DATABASE INTERNALS & IN-MEMORY ARCHITECTURES DEEP-DIVE ANALYSIS

Why DragonflyDB Uses B+ Trees Instead of SkipLists for Sorted Sets

Architectural breakdown based on the engineering dissection by Arpit Bhayani (Asli Engineering, Video ID: z-yC2o4GZvg).

> **THE FOUNDATIONAL ARCHITECTURAL DILEMMA**
>
> Redis has long defined in-memory data store performance by employing **SkipLists** for its **Sorted Set (`zset`)** data structure. However, **DragonflyDB**—a high-throughput, drop-in replacement for Redis offering up to 25x total throughput—radically diverges from this convention: it replaces SkipLists with an **in-memory B+ Tree**. By eliminating massive pointer overhead and exploiting modern CPU cache line geometry, DragonflyDB achieves a **40% reduction in memory consumption** and a **500% (5x) improvement in `zset` throughput**.

1. THE CONTEXT: HOW REDIS IMPLEMENTS SORTED SETS

The **Sorted Set** is one of the most widely used data structures in Redis. It underpins real-time gaming leaderboards, priority queues, sliding-window rate limiters, and time-series indexes. A Sorted Set enforces two strict guarantees:

- **Set Uniqueness:** Every member string is unique within the set.
- **Score Ordering:** Every member is bound to an IEEE 754 floating-point `score`, maintaining strictly ordered ranking.

Internally, Redis applies a dual-engine threshold based on element cardinality:

- **Small Sets (≤ 128 elements):** Redis encodes elements using a contiguous memory structure called `listpack` (formerly `ziplist`), where members and scores are stored sequentially in a single memory allocation to minimize overhead.
- **Large Sets (> 128 elements):** Redis promotes the set into a dual-structure combination: a standard **Hash Table (`dict`)** providing $O(1)$ member-to-score lookups, paired with a pointer-heavy **SkipList (`zskiplist`)** to support $O(\log N)$ range queries and rank updates.

2. SKIPLIST MECHANICS & THE "EXPRESS LANE" MODEL

A SkipList is a probabilistic multi-level data structure built upon a sorted linked list. While a standard singly linked list requires $O(N)$ sequential traversal for search, a SkipList adds hierarchical forward pointer layers on top:

- **Bottom Layer (L_0):** A doubly-linked list holding all member-score pairs sorted in ascending score order.
- **Express Lanes ($L_1 \dots L_k$):** Successive upper layers probabilistically skip nodes (typically with probability $p = 0.25$ or 0.5). Each layer above has roughly half the nodes of the layer beneath it.
- **Search Traversal:** Traversal starts at the highest express lane. The search steps forward until the target score is surpassed, drops down to the immediately lower lane, and repeats. This yields $O(\log N)$ search, insert, and delete complexity without the complex balancing tree rotations required by Red-Black or AVL trees.

3. IN-MEMORY SORTED SET IMPLEMENTATIONS COMPARISON

Technical Dimension	Redis SkipList (`zskiplist`)	DragonflyDB B+ Tree	Redis Listpack (Small Sets)
Primary Target	High cardinality sets (> 128 keys)	All large-scale Sorted Sets	Compact sets (≤ 128 keys)
Structural Overhead	**~37 bytes per entry**	**~3.2 bytes per entry (11x lower)**	Minimal (~1-2 bytes per entry)
Hardware Cache Locality	Poor (Dispersed heap pointer chasing)	Optimal (256-byte contiguous nodes)	High (Flat contiguous byte array)
Search & Range Time	$O(\log N)$ probabilistic traversal	$O(\log_B N)$ cache-conscious B-Search	$O(N)$ linear memory scan
Write & Mutate Throughput	Baseline (1x)	**Up to 5x single-thread, 24x multi-thread**	Degrades rapidly past 128 elements

4. THE POINTER OVERHEAD CRISIS IN REDIS SKIPLISTS

To understand why DragonflyDB rejected the SkipList, we must examine the C source code definition of a Redis `zskiplistNode`. A Sorted Set entry requires only two essential fields:

- **Member Pointer (`sds ele`):** 8 bytes (pointer to dynamically allocated string).
- **Score (`double score`):** 8 bytes (64-bit IEEE floating-point value).

The **bare minimum payload** required to store an entry is exactly $8 + 8 = 16$ bytes. Any additional memory allocated per node represents structural metadata overhead imposed by the data structure:

Redis `zskiplistNode` Overhead Breakdown

- **Backward Pointer:** 8 bytes (enables reverse iteration).
- **Forward Pointers:** 8 bytes per level.
- **Span Counter:** 8 bytes per level (tracks rank offset).
- **Level Array Overhead:** Each level requires $8 + 8 = 16$ bytes. Because levels are assigned geometrically ($p = 0.25$, up to 32 levels), empirical measurement across millions of nodes reveals an **average structural overhead of ~37 bytes per node**.
- **Total RAM Per Entry:** $16 ext{ bytes (data)} + 37 ext{ bytes (metadata)} pprox \mathbf{53 ext{ bytes}}$. Metadata consumes more than double the payload!

DragonflyDB B+ Tree Node Layout

- **Non-Binary Architecture:** High branching factor ($B \gg 2$) keeps the tree extremely shallow (2-3 levels max).
- **Fixed 256-Byte Leaf Nodes:** Leaf nodes do not store scattered heap pointers; they store flat arrays of contiguous entries.
- **Entries Per Leaf:** At 16 bytes per entry (8B member + 8B score), exactly 16 entries pack into a single 256-byte leaf node: $$ rac{256 ext{ bytes}}{16 ext{ bytes/entry}} = 16 ext{ entries per leaf}$$
- **Zero Per-Node Forward/Span Pointers:** Elements inside a leaf are sorted in-place, eliminating forward pointers and span tracking.

Figure 1: Architectural Comparison — Pointer-Chasing SkipList vs. 256-Byte Contiguous B+ Tree Leaf

```
REDIS SKIPLIST (37 Bytes Pointer Overhead per Node — Fragmented Heap Traversal):
[Node A: 16B Data] <---(8B Back)--- [Node B: 16B Data] <---(8B Back)--- [Node C: 16B Data]
  |-- Level 2: [Forward Pointer: 8B | Span: 8B] ----------------------> [Node C]
  |-- Level 1: [Forward Pointer: 8B | Span: 8B] ----------> [Node B] ---> [Node C]
  |-- Level 0: [Forward Pointer: 8B | Span: 8B] ----------> [Node B] ---> [Node C]

DRAGONFLYDB B+ TREE (Contiguous 256-Byte Chunk — Exactly 4 CPU Cache Lines):
[ 256-Byte Contiguous Leaf Node Array: 16 Packed Entries • 0 Pointer Overhead ]
| [M1:8B|S1:8B] | [M2:8B|S2:8B] | [M3:8B|S3:8B] | ... | [M15:8B|S15:8B] | [M16:8B|S16:8B] |
↑--- 64B L1 Cache ---↑--- 64B L1 Cache ---↑--- 64B L1 Cache ---↑--- 64B L1 Cache ---↑
```

5. THE MATHEMATICAL OVERHEAD BREAKDOWN (1,000-ENTRY MODEL)

STEP-BY-STEP MATHEMATICAL DERIVATION (ARPIT BHAYANI PROOF)

Consider an operational workload storing **1,000 entries** in a Sorted Set. Let us compare the exact memory allocation consumed by DragonflyDB's B+ Tree versus Redis's SkipList:

1. **Leaf Nodes Required:** Storing 1,000 entries at 16 entries per leaf requires: $$\lceil 1000 / 16 ceil pprox 63 ext{ to } 67 ext{ leaf nodes}$$
2. **Inner Routing Nodes Required:** Assuming a standard B+ Tree branching factor of 7 to 15, routing across ~67 leaf nodes requires approximately **10 inner nodes**.
3. **Total Node Allocation:** Total nodes = $67 ext{ (leaf)} + 10 ext{ (inner)} = \mathbf{77 ext{ total nodes}}$.
4. **Total Memory Footprint:** $77 ext{ nodes} imes 256 ext{ bytes/node} = \mathbf{19,712 ext{ bytes}}$.
5. **Average Memory Consumed Per Entry:** $$ rac{19,712 ext{ bytes}}{1,000 ext{ entries}} = \mathbf{19.2 ext{ bytes per entry}}$$
6. **Structural Metadata Overhead:** $$ ext{Overhead} = 19.2 ext{ bytes (total)} - 16.0 ext{ bytes (essential payload)} = \mathbf{3.2 ext{ bytes per entry}}$$

Conclusion: Redis incurs ~37 bytes of structural overhead per entry, whereas DragonflyDB incurs only 3.2 bytes—an unprecedented 91.3% reduction in metadata tax!

6. HARDWARE SYMBIOSIS: CPU CACHE LINES VS. POINTER CHASING

The advantages of DragonflyDB's B+ Tree extend far beyond raw byte savings; the data structure is fundamentally tailored to modern CPU microarchitectures:

- **The CPU Cache Line Bottleneck:** Modern x86-64 and ARM processors transfer data from L3 cache/RAM into CPU L1 caches in fixed chunks of **64 bytes** (cache lines). Retrieving data from main RAM requires 60–100 nanoseconds, whereas L1 cache hits execute in under 1 nanosecond.
- **SkipList Pointer Chasing:** SkipList nodes are scattered across arbitrary heap locations by the memory allocator (`jemalloc`). Traversing from Node A to Node B requires dereferencing pointers, causing consecutive CPU cache line misses and stalling the processor execution pipeline.
- **B+ Tree Cache Locality:** A 256-byte B+ Tree leaf node maps exactly onto **four 64-byte CPU cache lines** ($4 imes 64 = 256 ext{ bytes}$). Once the node header is read, the CPU prefetcher streams the entire 16-entry array into L1/L2 cache. Binary search across the 16 entries executes strictly in on-chip registers and L1 cache with near-zero memory stalls.

7. EMPIRICAL BENCHMARKS: REDIS V7 VS. DRAGONFLYDB

To validate the theoretical gains, DragonflyDB ran benchmarks firing over **1,000,000 `ZADD` operations** comparing Redis v7 against DragonflyDB across two distinct operational regimes:

Throughput Benchmark (Million `ZADD` Commands)

Test Condition	Redis v7 (1 Thread)	Dragonfly (1 Thread)	Dragonfly (8 Threads)
10 to 128 keys per ZADD	3.5k QPS	**12.0k QPS (~3.4x)**	**83.0k QPS (~24x)**
129 to 200 keys per ZADD	2.8k QPS	**10.5k QPS (~3.8x)**	**74.0k QPS (~26x)**

Key Finding: Even running on a single thread, DragonflyDB is ~3.5x to 4x faster than Redis due to cache-friendly search. Across 8 cores, Dragonfly's shared-nothing execution delivers 83k QPS.

Memory Consumption Scaling Characteristics

Cardinality Regime	Redis Behavior	DragonflyDB Behavior
Small Sets (≤ 128 elements)	Uses `listpack`; compact footprint; memory difference is minor.	Uses compact B+ Tree nodes; competitive with listpack footprint.
Large Sets (> 128 elements)	Escalates to SkipList + Dict; pointer metadata inflates memory by ~40-50%.	Maintains strict 256-byte leaf packing; saves **~40% overall RAM**.

Key Finding: As soon as cardinality exceeds the 128-element threshold, Redis memory explodes, while DragonflyDB scales linearly with minimal metadata overhead.

8. ARCHITECTURAL LESSONS & KEY ENGINEERING TAKEAWAYS

THE FIRST-PRINCIPLES SYSTEMS TAKEAWAYS

1. **The Fallacy of Algorithmic Big-O Equivalence:** On paper, both SkipLists and B+ Trees offer identical asymptotic complexity: $O(\log N)$ for search, insertion, and deletion. In production systems engineering, hardware invariants (cache lines, memory bus saturation, pointer indirection) dominate theoretical Big-O curves.
2. **Pointer Tax Accumulation at Scale:** In 64-bit architectures, pointers cost 8 bytes each. A data structure that requires forward pointers, backward pointers, and span counters per node incurs a massive "pointer tax" that dwarfs actual data payload size.
3. **Cache Line Alignment Outperforms Algorithmic Tricks:** Packing multiple records into fixed-size chunks (256 bytes) aligned to CPU cache boundaries eliminates cache misses and enables SIMD/vectorized instruction execution.
4. **Pragmatic Hardware Symbiosis:** DragonflyDB's achievement proves that radical performance improvements come from matching software data structures directly to the physical invariants of modern hardware.